PROJETO 1

Um sistema de informação geográfica simplificado, como definido abaixo.

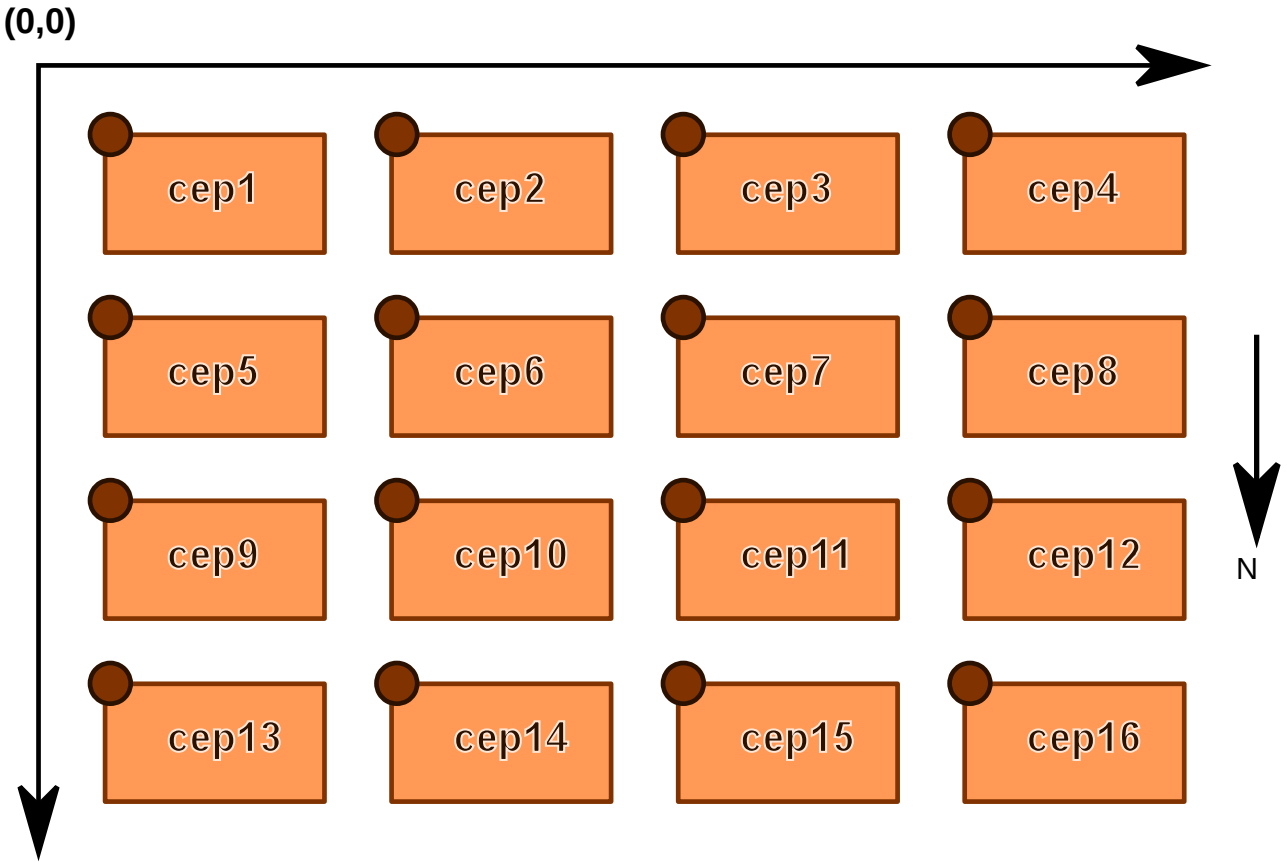
> *A geographic information system (GIS) is a computer system for capturing, storing, checking, and displaying data related to positions on Earth's surface. GIS can show many different kinds of data on one map. This enables people to more easily see, analyze, and understand patterns and relationships.*
> **Fonte: http://www.nationalgeographic.org/encyclopedia/geographic-information-system-gis/**

DESCRIÇÃO

Um Sistema de Informações Geográficas (SIG), para a nossa finalidade, é um sistema que contém (não exclusivamente) dados geo-referenciados, isto é, dados com algum atributo de localização espacial (uma coordenada).

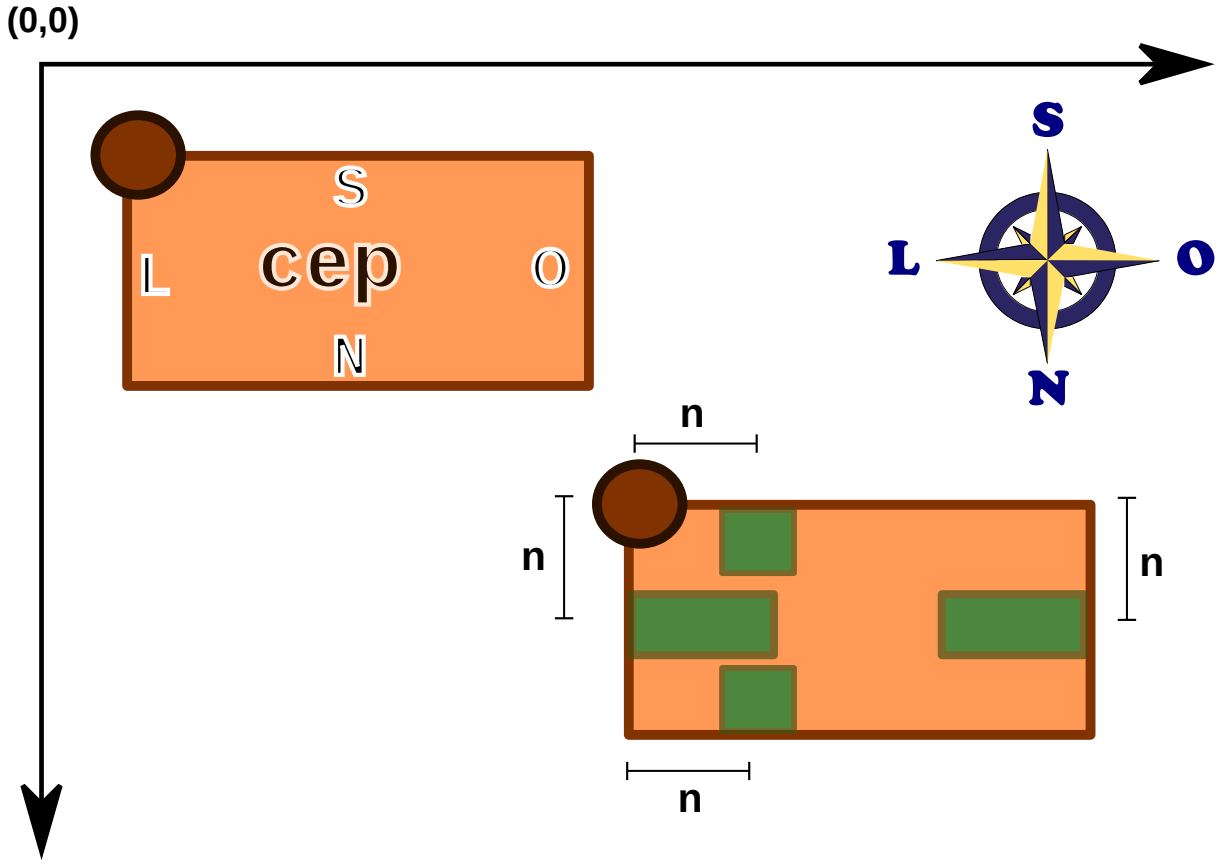
O sistema manipulará o mapa de uma cidade e algumas informações relacionadas.

O mapa de uma cidade é composto por um conjunto de retângulos que representam as quadras.



A cidade exemplificada acima chama-se **Bitnópolis** e possui 16 quadras. O sistema de endereçamento de Bitnópolis é inspirado no de nossa capital federal. Cada **quadra** possui 4 **faces** (N,S,L,O) e é identificada por um **CEP** alfanumérico. O número de uma casa ou estabelecimento

comercial é a **distância** da frente da casa até uma projeção do ponto de ancoragem do retângulo que representa a quadra (veja figura abaixo). Assim, um endereço é da forma CEP/Face/número, por exemplo, cep15/S/45. O ponto de ancoragem do retângulo é o canto sudeste da quadra.



Bitnópolis não é uma cidade deserta. Ela é habitada por pessoas. A maioria delas são moradores (moram em casas ou apartamentos), mas, infelizmente, também existem os habitantes sem-teto.

A Cidade

A cidade é descrita pelo arquivo **`.geo`**:

comando	parâmetros	
`q`	`cep x y w h`	*Insere uma quadra (retângulo e cep)*
`cq`	`sw cfill cstrk`	*Cores do preenchimento (`cfill`) e da borda (`cstrk`) das quadras, espessura da borda (`sw`) (a partir deste comando)*
Comandos do arquivo .geo		

Pessoas e moradores

Os habitantes de Bitnópolis estão relacionados num arquivo. Cada linha do arquivo contém a descrição de um habitante ou de um morador. Todo morador é habitante, mas nem todo habitante é morador. Portanto, os sem-tetos são os habitantes que não são moradores.

comando	parâmetros	
p	`cpf nome sobrenome sexo nasc`	*Insere habitante identificado por cpf, nomeada (nome,sobrenome), de um certo sexo (M,F), nascida numa determinada data (dd/mm/aaaa)*
m	`cpf cep face num compl`	*Informa que um dado habitante (cpf) mora num dado endereço (cep,face,num,compl)*
Comandos do arquivo .pm (-pm)		

Os comandos de atualização e consulta podem ser colocados em um arquivo `.qry`:[1]

comando	parâmetros	
rq	`cep`	*Remove quadra cujo cep é* `cep`*. Moradores da quadra passam a ser sem-tetos.* ***TXT**: informar cpf e nome dos moradores* ***SVG**: colocar um pequeno X vermelho no local da âncora da quadra removida.*
pq	`cep`	*Calcula o número de moradores da quadra (por face e total).* ***SVG**: colocar número de moradores de cada face (próximo ao limite da face) e, no centro da quadra, o número total de moradores da quadra*
censo		*Produz várias estatísticas sobre habitantes de Bitnópolis.* ***TXT**: reportar estatísticas: número total de habitantes, número total de moradores, proporção moradores/habitantes, número de homens, número de mulheres, % de habitantes homens, % habitantes mulheres, % de moradores homens, % de moradores mulheres, número total de sem-tetos, % sem-tetos homens, %sem-tetos mulheres*

1 O arquivo `.qry` será informado pelo parâmetro `-q`.

comando	parâmetros	
h?	cpf	*Dados sobre habitante identificado por cpf.* ***TXT:*** *reporta todos os dados sobre a respectiva pessoa. Se for morador, reportar também o endereço*
nasc	cpf nome sobrenome sexo nasc	*Pessoa nasce.*
rip	cpf	*Pessoa falece.* ***TXT****: reportar dados da pessoa falecida. Se for morador, reportar também o endereço.* ***SVG:*** *Colocar uma pequena cruz vermelha no local do endereço (se morador)*
mud	cpf cep face num cmpl	*Morador identificado por cpf muda-se para novo endereço.* ***SVG****: marcar o endereço de destino com um pequeno quadrado vermelho no local de destino. Colocar o cpf dentro do quadrado (usar fonte minúscula)*
dspj	cpf	*Morador identificado por cpf é despejado.* ***TXT****: reportar dados do habitante e endereço onde ocorreu o despejo.* ***SVG****: colocar um pequeno círculo preto no local do despejo.*
Comandos do arquivo .qry		

Implementação

Implementar Hashfiles Dinâmicos em disco (arquivo binário). O nome do arquivo relativo ao hashfile deve ter a extensão .hf (e .hfc, caso julgue necessário também manter um arquivo de cabeçalho separado). A final da execução também deve ser produzido um arquivo-texto (extensão .hfd), com uma representação esquemática, legível do conteúdo do arquivo .hf). Este arquivo-texto também deve registrar as expansões dos buckets. Os dados das quadras e dos habitantes devem ser mantidos em hashfiles, provavelmente em 2 hashfiles distintos.

Para cada módulo, escrever um arquivo de teste unitário usando o módulo unity (conforme descrição geral. O makefile deve conter targets para compilação e execução dos testes unitários.

Avaliação

Espera-se uma atitude pró-ativa para a aquisição dos conhecimentos (i.e., estudo) para resolver o problema proposto.

A avaliação consistirá da execução dos testes e da inspeção de código. A nota é dividida em 2 partes: 1.0, entrega do .h e testes unitários do hashfile dinâmico; 9.0, proporcional ao número de testes bem-sucedidos, aplicados os descontos escritos abaixo.

ATENÇÃO: Os focos do projeto são: implementação e uso da Estrutura de Dados proposta e modularização bem-feita. Hipoteticamente, se todo o resto estiver bem-feito e esta parte mal-feita, a nota será muito baixa.

ATENÇÃO: Caso algum tipo de FRAUDE seja detectada: Nota **ZERO** a **TODOS** os envolvidos.

Critério	Desconto
Definir struct em arquivo .h	2.5
Modularização Pobre: .h mal projetado, mal documentado	até 1.5
Não implementado conforme especificado • não implementar estruturas pedidas ou implementação não funcional (gravíssimo)	tipicamente até 4.5, mas em casos graves, o desconto pode ser total
Procedimentos extensos e/ou complicados	até 1.0
Makefile não funcional	Caso não produza executável: veja "Erro de Compilação"
Poucos commits ou commits concentrados perto do final do prazo.	tipicamente até 3.0; porém, o padrão de commits pode indicar possível fraude.
Implementação ineficiente e/ou desidiosa.	Até 1.5.
Módulos sem respectivos arquivos de testes unitários, ou testes unitários mal-feitos.	Até 2.5
Makefile sem targets (funcionais) para produzir e executar os casos de teste	Até 1.5
Erro de compilação	Nenhum teste será executado. Portanto, nota Zero, especialmente se for causado por negligência do aluno.

O Que Entregar

Os arquivos fontes devem estar em um repositório GIT. Submeter ao Classroom um arquivo-texto (.txt) com uma linha, com o formato abaixo. A URL fornecida será usada para clonar o repositório (git clone)

`apelido url-do-repositorio`
Exemplo: `joseMane https://github.com/zemane/t2.git`

ATENÇÃO:
* os fontes devem ser compilados com a opção `-fstack-protector-all`.
* adotamos o padrão C99. Usar a opção `-std=c99`.

RESUMO DOS PARÂMETROS DO PROGRAMA TED

Parâmetro / argumento	Opcional	Descrição
`-e` ***path***	S	Diretório-base de entrada (**BED**)
`-f` ***arq*****.geo**	N	Arquivo com a descrição da cidade. Este arquivo deve estar sob o diretório **BED**.
`-o` ***path***	N	Diretório-base de saída (**BSD**)
`-q` ***arqcons*****.qry**	S	Arquivo com consultas. Este arquivo deve estar sob o diretório **BED**.
`-pm` ***arq*****.pm**	S	Arquivo das pessoas e moradores